API Gateway Explained

what belongs in it, BFFs and avoiding a SPOF

Watch the 4-min video

In one line: an API gateway is one front door that handles the cross-cutting work every request needs (TLS, auth, rate limits, routing, observability); keep business logic out of it, and run it as redundant stateless instances so it never becomes a single point of failure.

1 Why one front door

Your company runs twenty Python microservices: orders, users, payments and more. Mobile apps and partners need to call them. Do you expose twenty hostnames, each doing its own TLS, authentication and rate limiting? No. You put **one front door** in front of them: an **API gateway**.

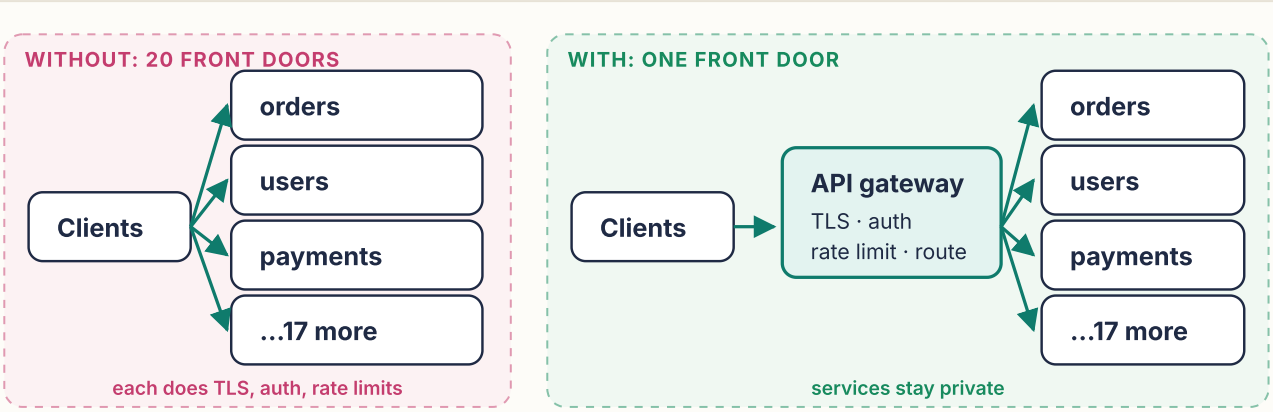


Figure 1. Without a gateway every service repeats the same edge work and is exposed to the internet. With one, the services stay private behind a single hostname.

2 What a gateway does

A gateway is a single entry point for the **cross-cutting concerns**: the jobs every request needs regardless of which service handles it.

- **Routing** by path or host: `/orders` goes to the orders service.
- **TLS termination**: certificates live in one place.
- **Authentication**: validate API keys or JWTs (signature, expiry, audience) and check scopes.
- **Rate limits and quotas** per key, user or partner.
- **Transformations**: add headers, or translate REST to gRPC.
- **Observability**: logs, metrics and traces for every call, in one consistent format.

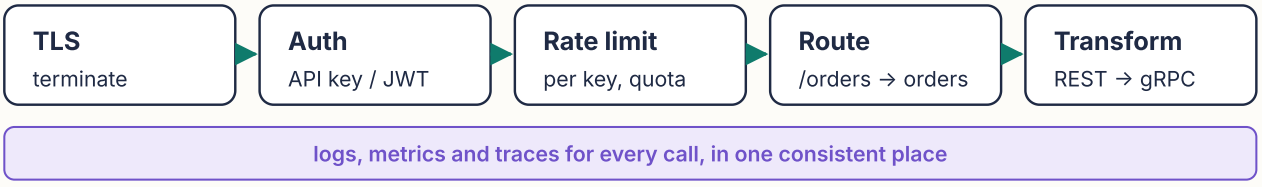


Figure 2. The typical policy chain. Each step is generic: it works the same for every API behind the gateway.

Aggregation and the backend for frontend (BFF)

Some gateways also **aggregate**: one client call fans out to several services and the results are merged, so a mobile screen needs a single round trip instead of five. This often lives in a separate **backend for frontend**, one per client type (mobile BFF, web BFF), rather than in the shared gateway. The BFF is owned by the team that builds that client, so it can change at their pace without touching the platform.

3 A concrete setup: Azure API Management

Clients call one public hostname. **Azure API Management** validates the JWT issued by **Entra ID**, applies a rate limit per subscription key, and routes to **FastAPI** services running privately, inside a virtual network or in Kubernetes behind an internal load balancer. Developers also get a portal with docs and keys. Similar options: **Kong**, **AWS API Gateway**, **Apigee**, and **Envoy**-based gateways.

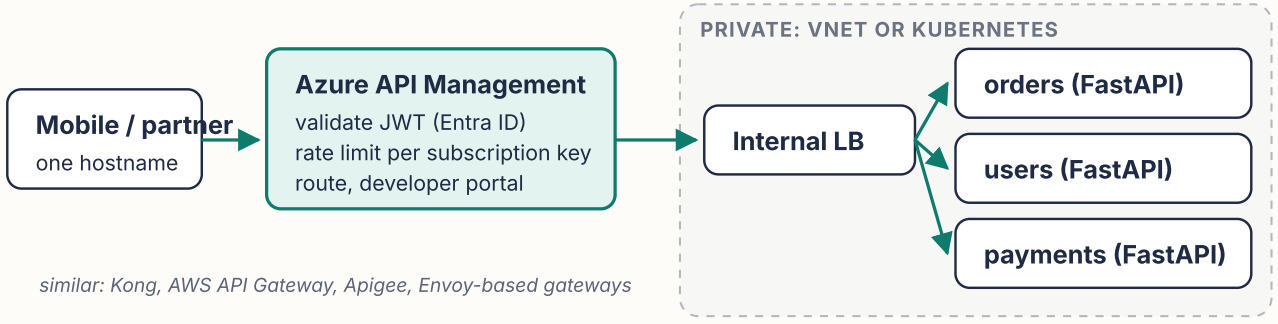


Figure 3. The gateway is the only public piece. The services have no public IPs; they trust only traffic that came through it.

4 What does not belong in the gateway

Business logic. Order validation, pricing rules, orchestrating business workflows. Also **object-level authorization**, because that needs the data (lesson SD-C-016): the gateway can check “has orders:read”, not “owns order 1002”.

Once teams push domain logic into the gateway, you get a **god gateway**: every change waits on the platform team, it becomes a release bottleneck, and one bad config breaks every API at once. Keep it **thin**, with generic policies only.

RULE OF THUMB

If a rule would be the same for any API (validate a token, limit to 100 requests/min, add a trace ID), it can go in the gateway. If it mentions a business noun (an order total, a discount, a stock level), it belongs in a service.

5 Don't let it become a SPOF

The gateway sits on **every** request, so if it goes down, everything goes down.

- Run **multiple stateless instances** across availability zones, behind a load balancer.
- Keep **shared state**, like rate-limit counters, in an **external store**, and decide in advance what happens when that store is down (allow traffic, or reject it).
- Use a **zone-redundant managed tier**, or multiple regions with **DNS failover**.
- Roll out **config changes gradually**, with canaries and quick rollback: a bad route or policy hits every API.
- Watch the gateway's **own latency and errors**: it adds a hop to every call.

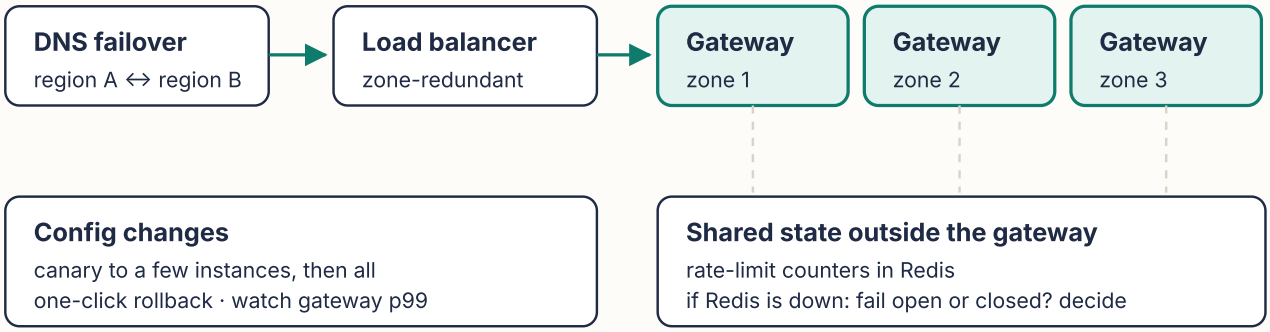


Figure 4. Redundant at every level: regions, zones and instances. The instances share no memory, so any one can serve any request.

Don't confuse it with its neighbours

Component	What it does	Traffic
Load balancer	spreads connections across servers; knows little about APIs	any
Reverse proxy (NGINX)	forwards requests; can do some gateway work (TLS, routing)	any
API gateway	API-aware policies: keys, JWTs, quotas, transforms, developer portal	north-south (outside → in)
Service mesh	mutual TLS, retries and telemetry between services	east-west (service ↔ service)

COMMON MISTAKES

- Putting pricing, validation or workflows in gateway policies (the god gateway).
- Trusting the gateway for object-level authorization.
- A single gateway instance, or rate-limit counters held in one instance's memory.
- Pushing a config change to all instances at once, with no rollback.
- Services still reachable from the internet, bypassing the gateway.

6 Key terms

API gateway: single entry point for cross-cutting API concerns

BFF: backend for frontend, one per client type

North-south: traffic from outside into the system

Cross-cutting concern: work every request needs: auth, TLS, limits

God gateway: gateway overloaded with business logic

East-west: traffic between internal services

7 Interview questions

Q1. What belongs in an API gateway, and what doesn't? BASIC

Cross-cutting concerns: routing, TLS, authentication, rate limiting, transformations and observability. Not business logic, domain validation or object-level authorization; those stay in the services.

Q2. API gateway vs load balancer vs service mesh? BASIC

A load balancer spreads connections and knows little about APIs. A gateway applies API-aware policies to north-south traffic from outside. A mesh handles east-west traffic between services with mutual TLS and retries.

Q3. What is a backend for frontend? BASIC

A thin API layer per client type (e.g. mobile, web) that aggregates several service calls into the shape that client needs, so one screen needs one round trip. It is owned by the client team, not the platform team.

Q4. How do you keep the gateway from being a single point of failure? SCENARIO

Several stateless instances across zones (or regions with DNS failover) behind a load balancer; shared state such as rate-limit counters in an external store with a clear fallback; gradual config rollout with canaries and rollback; and alerts on gateway latency and errors.

Q5. A team wants the gateway to reject orders over $10,000. Good idea? SCENARIO

No. That is a business rule; it needs domain data and changes with the business. Put it in the orders service. Otherwise every rule change waits on the platform team and one bad policy can break every API.

Q6. Rate-limit counters live in Redis and Redis goes down. What should the gateway do? SCENARIO

Decide in advance. Fail open (allow traffic, maybe with a coarse local per-instance limit) to keep the product up, or fail closed for expensive or abuse-prone endpoints. Never block on Redis with no timeout.

Q7. The gateway adds 15 ms to every call. Is that a problem? TRICKY

It depends on the SLO, but it is paid on every request, so measure it. Keep policies cheap (cache JWKS keys, avoid synchronous calls to other services in policies), keep the gateway in the same region as the services, and track its own p99 separately.

Q8. The gateway already validated the JWT. Can services skip all checks? TRICKY

They can trust the gateway for token validity if the network guarantees all traffic passes through it, but they must still do object-level authorization, which needs their data. Many teams also re-verify the token in the service as defense in depth.

Next: SD-C-018 · Retries, Exponential Backoff & Jitter backendroadmap.com